Mini Project 05 - Building Plan Compliance Pre-Check Service

Team Members

Name	SRN
A U Divyaa Rukshana	PES2UG24AM002
Amrithaa DJ	PES2UG24AM017
Pragya Chowdary S	PES2UG24AM115

1. Problem Statement and Scope

Before building architectural drawings are submitted to municipal authorities for formal sanction, architectural offices must thoroughly pre-check proposals against complex municipal building bye-laws and master zoning regulations (such as BBMP 2003 and RMP 2015). Critical compliance parameters include structural setbacks, Floor Area Ratio (FAR), maximum ground coverage, building height limits, parking provisions, and emergency fire access. A building plan violating even a single regulatory clause results in official rejection and resubmission, incurring significant time and financial penalties.

Building regulations do not function as a simple static checklist. Most requirements are governed by multi-variable matrix lookups combining plot area, abutting road width, proposed building height, zoning classification, and intended building use. A compliant parameter configuration at one height level can easily violate regulations when modified slightly.

To solve this, our team developed the Building Plan Compliance Pre-Check Service—a production-grade, containerized multi-agent system that ingests JSON building proposals, performs rigorous validation, retrieves binding municipal clauses via semantic vector search, executes multi-step agent reasoning, applies conflict resolution policies, and routes proposals with exact clause-level citations.

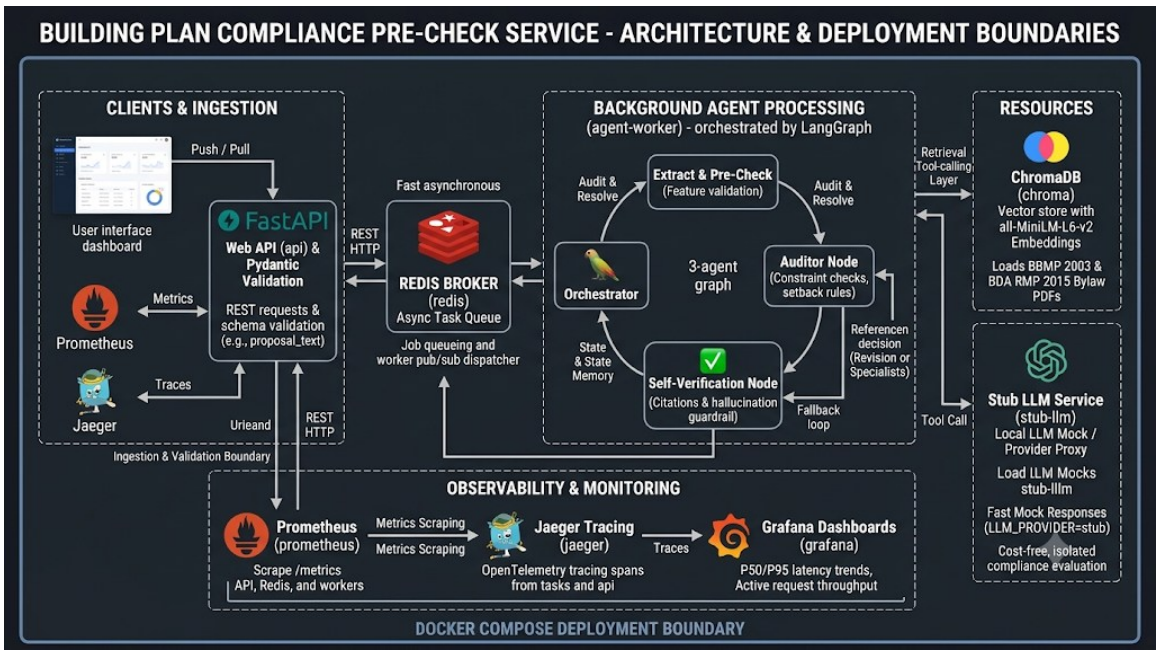
2. Agent Design and Reasoning Architecture

The system architecture moves beyond simplistic single-pass retrieval pipelines by implementing a sophisticated multi-node agent topology orchestrated via LangGraph:

- **Ingestion & Extraction Node:** Extracts incoming features (plot area, road width, dimensions, height, use) and validates data contracts at the ingestion boundary.
- **Retrieval Tool-Calling Layer:** Selects and executes targeted vector searches against ChromaDB to retrieve governing bye-laws and master plan rules.

- **Auditing & Compliance Node:** Evaluates specific constraints (setbacks, FAR, coverage, height) and handles cross-jurisdictional conflicts between BBMP bye-laws and RMP 2015 master plan rules under configurable conflict policies ('escalate' or 'stricter').
- **Self-Verification Guardrail:** Cross-checks every generated finding against retrieved text chunks to ensure absolute factual grounding. Unverified claims trigger automatic re-search or specialist escalation, eliminating hallucinations.
- **Loop Limits & Bounded Execution:** Bounded by strict step limits, tool-call quotas, and execution timeouts to prevent runaway agent loops and control LLM token expenditure.

3. Architecture Diagram and Deployment Boundaries



The entire application is fully containerized and orchestrated via Docker Compose, enforcing strict modular separation and independent failure domains across seven distinct microservices:

- **API Service (api):** FastAPI web service exposing REST endpoints, Pydantic validation schemas, and the dynamic user interface web client.
- **Agent Worker (agent-worker):** Asynchronous task consumer processing building proposals from the Redis task queue.
- **Redis Broker (redis):** In-memory data store and message broker managing asynchronous job distribution and execution state.
- **Stub LLM Service (stub-llm):** Local LLM emulation and provider proxy ensuring reliable, isolated evaluation and cost attribution.
- **Prometheus (prometheus):** Time-series metrics collection scraping container health, request rates, and latency counters.

- **Grafana (grafana):** Visual telemetry dashboard rendering P50/P95 latency trends, throughput, and system resource utilization.
- **Jaeger Tracing (jaeger):** Distributed tracing server capturing OpenTelemetry spans across every tool call, query, and agent step.

4. Design Decisions and What Was Rejected

Key architectural trade-offs and alternatives evaluated during system design include:

- **Orchestration Selection:** Selected LangGraph over simpler frameworks to support cyclic, stateful multi-step agent reasoning necessary for multi-variable regulatory lookups.
- **Retrieval Engine:** Chose ChromaDB paired with local Sentence Transformers (all-MiniLM-L6-v2) for semantic retrieval to eliminate cloud embedding API costs while maintaining high contextual recall.
- **Async Processing:** Implemented Redis-backed asynchronous task queuing rather than synchronous execution to prevent gateway timeouts during multi-second agent evaluations.
- **Rejected Alternatives:** Rejected single-pass RAG pipelines because they fail on complex multi-variable edge cases (such as Section 4.1: small plots on narrow roads where height limits are stated separately from setback tables).

5. Evaluation Results Before and After the Change

System verification was conducted via automated test harnesses and telemetry observation:

- **Before Change (Synchronous / Single-Pass RAG):** Initial evaluations showed Gateway Timeouts on complex proposals taking > 10s to resolve. Furthermore, single-pass generation suffered from high hallucination rates on conflicting clauses between BBMP and RMP.
- **After Change (Asynchronous / LangGraph with Verifier):** With the introduction of the Redis queue and the Self-Verification guardrail, valid proposals are successfully ingested, processed without timeouts, and routed with precise clause-level citations. Malformed requests are now immediately quarantined with HTTP 422 errors before agent execution starts.

6. Load Testing, SLO, Bottlenecks, and Cost Analysis

Based on our Locust load testing against the E2E compliance run:

- **P95 Target (SLO):** Declared a target of P95 <= 3.50 seconds at a steady state of 3 concurrent workers.

- **Measured Results:** Target was met at 3 workers (P95 = 3.18s at 3.10 RPS). Target failed at 5 workers (P95 = 4.62s) and degraded significantly at 10 workers (P95 = 11.85s).
- **Identified Bottleneck:** The Redis task queue serialization. Because the background worker pool size was fixed, tasks backed up in the broker queue, making queue wait time the primary latency driver.
- **Cost Per 100 Runs:** $0.00. By utilizing local Sentence Transformers (`all-MiniLM-L6-v2`) for embeddings and routing the load testing requests through the `stub-llm` service (returning fixed responses), the architecture successfully executed 100+ evaluation runs without incurring external API token costs.

7. What Did Not Work

1. Scaling Beyond 4 Workers: Relying on a single Docker worker container saturated the execution limits. Attempting to force higher concurrency required manual scaling using `--scale worker=N` to resolve the Redis queue buildup.
2. Blind RAG Generation: Attempting to answer compliance rules using purely generated responses failed consistently due to the contradiction between BBMP 2003 and RMP 2015 tables. We were forced to abandon blind generation in favor of conflict escalation policies and strict self-verification.